# Chapter 5 | Programming in C / C++ / Java (OOP)

## 5.1 C Basics — Tokens, Identifiers, Variables, Storage Classes

C was created by **Dennis Ritchie** at Bell Labs (C++ — Stroustrup; Java — Gosling). **FORTRAN = Formula Translation** (asked twice). A **source program is written in a high-level language** by the programmer; the compiler turns it into the object program. **Syntax is the structure or format of data; semantics is its meaning. C tokens are BOTH the smallest individual units of a C program AND the basic elements recognised by the compiler.**

**(1) Identifier rules**: letters, digits and underscore only; may **begin with a letter or underscore, never with a digit or any other special symbol** (123Variable and 'my Variable' are invalid; _myVariable is fine). An identifier cannot be a keyword — **void is an invalid variable name**; in Java, **static** is likewise invalid.

**(2) Keywords**: auto, sizeof, signed, **volatile** are C keywords; "variable", "automatic", "integer" and "Constant" are NOT (the real words are auto, int, const).

**(3) char x;** declares a character variable. A number **with a decimal point is a fractional (floating-point) number**. In **int a, b, c = 10; only c is initialised** — a and b hold garbage, because an **uninitialised variable contains an indeterminate (garbage) value**, never an automatic 0.

**(4)** A variable visible only inside one module/block has **local scope** (block scope: {} to }); a **global** variable is visible throughout. Chained assignment **a = b = c = 15 sets all three; c = c + 10 then gives 25.**

| Storage class | Storage / default value / scope |
|---|---|
| **auto** | Memory; **garbage** default; **scope local to the block where defined** |
| **register** | **CPU register**; **default value is GARBAGE, not zero** — the 'default 0' claim is the tested incorrect statement |
| **static** | Memory; **default value 0**; retains value between calls |
| **extern** | Memory; default 0; **global — accessible by all functions**, declared with extern elsewhere (all three tested facts true) |

**"Intern" is not a storage class.** The **preprocessor** runs before compilation and does all three tested jobs: **replaces every #include with the file's contents, handles escape/character processing, and expands macros** — so #include is resolved **during preprocessing** (preprocess → compile → assemble → link).

**Worked:** Memory of struct { short a[5]; union { float v; long z; } u; } with short = 2, float = 4, long = 8: struct adds members (5×2 = 10), the union takes its **largest** member (8) → **18 bytes**.

### Exam Points — 5.1

- ✦ C = Ritchie; FORTRAN = Formula Translation. Tokens = smallest units + elements the compiler recognises. Syntax = format, semantics = meaning; source program = high-level.
- ✦ Identifiers: letters/digits/underscore, never digit-first or special-first; void invalid; volatile is a keyword, 'variable' is not. Only c in int a, b, c = 10; uninitialised = garbage.
- ✦ Storage classes = auto/register/static/extern (no 'Intern'); register lives in a CPU register but defaults to garbage; static/extern default 0; extern = global + all functions.
- ✦ Preprocessor: includes + escapes + macros (all three), before compilation. Struct-with-union worked = 18 bytes. a = b = c = 15 → c + 10 = 25.

## 5.2 C Operators, Precedence and Expression Tracing

**(1) Parentheses ( ) have the highest precedence of all**; among arithmetic operators **%** (together with * and /) outranks **+ and −**, and = sits lowest. **!= is a relational operator** (&&, || and ! are logical — so **&& is NOT arithmetic**). **|| returns true if at least ONE operand is true** ("both" is the false claim).

**(2) ?: is the only ternary operator — exactly 3 operands** (the '4 operands' claim is false), and **+ may be unary or binary** (both roles). **Assignment: symbol =, RIGHT-associative** — as is ?: — so x ? y : a ? b : c with x = 0, a = 0 → **c**; and the grade ladder (marks = 75) → **'B'**.

**(3) Compound assignment brackets the whole right side**: x −= y + 1 means x = x − (y + 1) = **x − y − 1**; **+= adds the right operand to the left and stores it back**. The **comma operator keeps the right value**: k = (v = 20, v + 2) → **22**.

**Worked:** x = a + b*(c−d)/e + f%g with 8,5,10,3,2,7,3 → 8 + 5×7/2 + 1 = 8 + 17 + 1 = **26** (integer division). **7 >= 7 prints 1** (C prints 1, not TRUE). !(5>3) && (8<10) = false && true = **FALSE**. **% takes the sign of the DIVIDEND**: 7%−3 = **1**, −7%3 = **−1**. Bitwise: **4 ^ 3 = 7** (XOR); Java **5 & 7 = 5** (AND).

**(4) Increment tracing**: C x = 5, y = x++ + ++x → 5 + 7 = **12**; Java x = 6, x++ + ++x → 6 + 8 = **14**; k = ++val * ++val (val = 1) → 3×3 = **9** (per key; modifying twice in one expression is formally undefined); k/++v with 8/9 → integer **0**; Java while (++m < −−k); from 1000/3000 → they meet at **2000**.

**(5) printf returns the number of characters printed**: printf("%d", printf("C Programming")) → **C Programming13**; "Tata Consultancy" → **16**. **Number bases in literals**: 62 (decimal), 062 (octal = 50), 0x62 (hex = 98) print **625098**; Java's 017 = **15**.

### Exam Points — 5.2

- ( ) highest; % beats + −; = lowest and right-associative; != relational; && logical; || = at least one true; ?: = 3 operands, right-associative (→ c; → 'B').
- x −= y+1 = x−y−1; comma → 22; precedence worked 26; 7>=7 → 1; %-sign follows dividend (1, −1); 4^3 = 7; 5&7 = 5.
- x++ + ++x: 12 (C, from 5) and 14 (Java, from 6); ++val*++val = 9; 8/++9 = 0; ++m<--k → 2000. printf returns count (13; 16); 62,062,0x62 → 625098; 017 = 15.

## 5.3 C Control Flow — if-else, switch, Loops

**(1) if syntax**: if (condition) { } else { } — no 'then'. The **if statement makes decisions**; in an **if-else ladder, once a condition is true NONE of the subsequent conditions are evaluated**. 5 > 10 false → prints the else line. **Dangling else: an else binds to the NEAREST unmatched if** (if#2), whatever the indentation.

**(2) switch = the multi-choice decision statement**. Control **falls through successive cases until a break or the end of the switch**: case 5 with no break → **FiveDefault**; nested switches are **allowed** (a switch may sit inside any case) — the nested working prints **Case 3Outer Default** (inner break, outer fall-through). A statement after the switch's unconditional exits is **unreachable (a4)**.

| Loop | Behaviour |
|---|---|
| **while** | **Pre-test**: condition first — **false at the start ⇒ the body never runs**; while(1) and while(5) are **infinite** (any non-zero value is true; only 0 is false) |
| **do-while** | **Body FIRST, condition after** (statement Q only — P reverses it, and the body CAN be a block). So it always runs at least once: i = 5 with while(i>5) prints just **5**; 1–5 prints **1 2 3 4 5** (the source key adding 6 is corrected); Java 10–12 prints three lines |
| **for** | **All three parts are optional — omitting the condition makes it infinite**; for(i=0;i<5;i++) runs **5 times** (i = 0…4); a loop that never terminates = **infinite loop**; there is **no foreach in C** |

**(3) break terminates the loop immediately; continue only skips to the next iteration**. Nested loops multiply: 3 × 2 = **6** prints; the star triangle prints *, **,** *. Extra increments inside the body: i++ in both header and body → **1 3**; if (++i % 2 == 0) printf → **2 4 6 8 10**.

DSSSB – TGT COMPUTER SCIENCE ( PRACTICE KING ) BOOK

Call/Message: 7011121290 Available @ Amazon: https://amzn.in/d/07E04o9B

One counting result worth memorising — doubling loop for (j = 1; j <= n; ) j = j * 2 makes

$$comparisons = log_2 n + 1$$

**Note:** Strays parked here from other chapters: the linked-list table questions — traversal from location 0 gives **A, C, E, X, B**, and inserting X after E relinks only E→X and X→(old successor) (option a); queue ends — insertion at REAR, deletion at FRONT, so both reversed statements are wrong (full treatment in Data Structures). Also **arr[i] = i²−2i+1 = (i−1)² → 1 0 1 4 9**.

### Exam Points — 5.3

- ✦ if(cond){}else{}; ladder stops at the first true; dangling else → nearest if. switch = multi-choice; fall-through till break; FiveDefault; Case 3Outer Default; nested switch legal; a4 unreachable.
- ✦ while pre-tests (false start → zero runs; while(1)/while(5) infinite); do-while = body first, at least once (5; 1 2 3 4 5; 10–12); for parts all optional (no condition → infinite); 5 iterations; no foreach.
- ✦ break exits, continue skips; 3×2 = 6; star triangle; '1 3' and '2 4 6 8 10' increment traps; doubling loop → $\log_2 n + 1$ comparisons; (i−1)² array; linked-list traversal A,C,E,X,B.

## 5.4 C Functions, Recursion and Scope

**Function prototype:** tells the compiler **the number and types of the arguments and the return type** — the interface only, never the body, and it is placed BEFORE use. Calling func(10) when the prototype says func(void) is a **compilation error** (call must match). **Nested function definitions are not allowed in standard C** — some compilers (GCC) accept them only as an extension.

**(1)** **Actual parameters are the values passed to the function when it is called**; formal parameters appear in the definition. **Call by value copies** — to modify the caller's variable, **pass its ADDRESS**. The pointer-swap program prints Before *p1=10 *p2=20, After **20/10**; the array passed to modify() is changed in the caller — data[2] becomes **6** (arrays travel by address). But **reassigning the pointer parameter itself (p = &q) stays local** — the caller still prints **20**.

**(2)** **Recursion = a base condition + a recursive call**; the procedure calls itself until the base condition. With a **static** local (initialised once, never reset): f(1) with static i → n grows 1→2→4→7 → returns **7**.

**(3)** **Inner-block declarations shadow outer ones only inside that block**: outer x=10, y=20; inner y=40 incremented; result **x = 11, y = 20**.

**(4)** Reading declarations inside-out: pointer-to-float and pointer-to-pointer-to-char in, pointer-to-pointer-to-int out = int ∗ ∗ fun(float ∗ , char ∗ ∗ ) — each pointer level adds one star (written here with the ∗ glyph; in code it is the ordinary asterisk). **Variadic (variable-argument) functions come from <stdarg.h>** (va_list/va_start/va_arg; the source key saying stdio.h is corrected). On a **CALL instruction the stack pointer DECREASES by 2 (or 4 on 32-bit)** — the return address is pushed and the stack grows downward.

### Exam Points — 5.4

- ✦ Prototype = arg count + types + return type only; func(void) called with an argument fails to compile; nested functions non-standard.
- ✦ Actual parameters = values at the call. Address-passing modifies the original (swap 20/10; data[2] = 6); reassigning the received pointer is local (20). Recursion = base + self-call; static-i recursion returns 7.
- ✦ Inner block shadows (x = 11, y = 20). Declaration reading: int-star-star fun(float-star, char-star-star). <stdarg.h> for variadic. CALL → SP decreases by 2/4.

## 5.5 C Arrays, Strings and Pointers

**(1)** Declaration: **int array[15];**. 2D: **int arr[2][2] = {1,2,3,4}; is valid** (only the first dimension may be omitted; commas inside one bracket are wrong). **An array name cannot be assigned after declaration** — char arr[6]; arr = "consultancy"; is the invalid form (string literals attach only at declaration).

**(2)** arr[i] ≡ *(arr + i) — the core identity; both print element i (the '3 3' question). For arr[2][2] = {{1,2},{3,4}}: *(*(arr+1)+1), *(arr[1]+1) and arr[1][1] all give **4**, but *(*arr+1) gives arr[0][1] = 2 — the odd one out. **sizeof of the whole array int arr[5] is 20** (whole array, not the pointer).

**(3)** **Pointer arithmetic on strings**: printf(str + 7) on "Hello, World!" → **World!**; char *S = "MySQL"; S += 2 → **SQL**; printf(6 + "Hello World") → **World** (the 'compilation error' key is corrected); a char* prints from where it points to the null. Array-pointer mix: N[] = {1,2,3,4}, P = N, N[1] += 10 → prints **1 : 12**. str[2][4] of {"one","two","three"} = final '**e**' of "three".

**(4)** int *arr[5] = an ARRAY of 5 pointers to int; int (*arr)[5] = a POINTER to an array of 5 ints — **parentheses change the binding**. A **NULL pointer signifies the pointer references no valid memory** (and helps avoid dangling pointers). **Static array: fixed size, stack; dynamic array: resizable at runtime, heap.**

**(5)** **Strings may contain letters, numbers AND symbols** (all three). **%4s reads at most 4 characters** ("HelloWorld" → **Hell**); **sprintf writes into the buffer and returns the character count → 13 Hello, World!**; **cgets() reads a console string**; **scanf lives in <stdio.h>** — and scanf("%d",&val) ≡ **fscanf(stdin, "%d", &val)** (printf ≡ fprintf(stdout,…)). The self-referential node: struct node { int data; struct node *next; }; — **the pointer inside is of the struct's own type**.

### Exam Points — 5.5

- ✦ int array[15]; arr[2][2] = {1,2,3,4} valid; array names not assignable. arr[i] = *(arr+i) (3 3); *(*arr+1) = 2; sizeof = 20.
- ✦ str+7 → World!; S+=2 → SQL; 6+"Hello World" → World; 1 : 12; str[2][4] = e. int *arr[5] vs int (*arr)[5]. NULL = points to nothing. Dynamic = heap + resizable.
- ✦ Strings hold letters + digits + symbols; %4s → Hell; sprintf returns 13; cgets() console; scanf = fscanf(stdin); struct node self-referential pointer.

## 5.6 C Structures, Unions and File Handling

**(1)** **Structure member access uses the dot**: book.price, book.page (arrow -> for a pointer to the structure). Struct memory adds members; a union takes its largest (the 18-byte worked case in 5.1).

| File mode / call | Asked meaning |
|---|---|
| **"wb"** | **Write in BINARY mode** (w = write, b = binary) |
| **"a+"** | **Read AND append** (w+ reads/writes but truncates; r reads only) |
| fopen on an empty, readable file | Returns non-NULL → prints **Success** (emptiness is irrelevant to opening) |
| **fseek(fp, 0, SEEK_SET)** | **Rewinds to the beginning** — re-reading input.txt gives **42** again |
| fwrite / fseek / **ftell** | fwrite writes a block ✓; fseek sets the position to a byte ✓; **ftell REPORTS the current position — it does not move to the start** (only fwrite + fseek correct) |
| **fclose()** | Closes a file opened with fopen |

### Exam Points — 5.6

- ✦ Dot for objects, arrow for pointers (book.price). "wb" = binary write; "a+" = read + append; empty readable file still opens (Success).
- ✦ fseek SEEK_SET rewinds (42); ftell only reports position; fclose closes.

## 5.7 OOP Concepts — The Four Pillars

**Object-Oriented Programming pays more attention to DATA than to functions** (procedural/structured programming is function-centric). All of **encapsulation, inheritance and polymorphism — with abstraction — are required pillars**; a language supporting just one is not purely object-oriented. **Data hiding and message passing are OOP concepts, but platform independence is NOT** (that comes from a virtual machine). **Global variables and function overloading**: overloading, encapsulation and abstraction are OOP features — **global variables are not**; "abduction" is a distractor. OOP languages: **LISP, Java, C++, Simula** (Simula was the first; SQL and C are not). OOP's **major goal is REUSE**, and its common home is **mobile applications**.

| Pillar | One-line identity |
|---|---|
| **Class / Object** | Class = **blueprint** — a **user-defined datatype with data members and member functions**; object = **instance of the class** (created with **new**); the variable referring to an instance is the object. Attributes = the data members (class student → **name and marks**; S1, S2 are objects) |
| **Encapsulation** | **Wrapping data and the functions that operate on it into a single unit** — what a class encapsulates is **function and data**; wrapping characteristics + behaviour = data encapsulation |
| **Abstraction** | **Representing essential features without the background details** — hiding; removing detail from a representation |
| **Inheritance** | **Parent-child (is-a) relationship** for **code reusability**; Animal → Cat is the valid example; it reflects **generalisation**; creating the new class = **derivation**; features = sub class + super class + reusability (operator overloading is polymorphism's) |
| **Polymorphism** | One entity, many forms — an OOP property (operator overloading lets an instance take many forms) |

**(1)** **Objects interact by SENDING MESSAGES** — myObject.doActivity() = **calling the method doActivity on the object myObject**. **Data hiding** = the compile error you get when private data is touched from outside. A language able to **create new data types is EXTENSIBLE**. **Modularity** = dividing a program into independent parts.

**(2)** **Constructor**: called **whenever an OBJECT is created** (never on mere class declaration); **has NO return type** (the C#.NET false statement); the **compiler provides the DEFAULT constructor** if none is defined; it must be **public** for objects to be creatable in any function (a private constructor is the singleton trick). **Copy constructor = initialises an object with the values of another object** of the same class — STOCK worked: S2(10) → Function 2 (one float), STOCK S2; → default, STOCK S1(S2) → copy. A class **with no name (anonymous)** is allowed but — keyed — **cannot have a destructor** (nothing to name it by).

**(3)** **Destructor** (~) deallocates; it is **mandatory when objects are created DYNAMICALLY** (own resources must be released). Match: **overriding = changing an inherited method's definition; overloading = one name, many definitions; destructor = deallocation function** (i-3, ii-1, iii-2).

### Exam Points — 5.7

- ✦ OOP = data-centric; four pillars all required; data hiding + message passing in, platform independence out; global variables/abduction not OOP; LISP/Java/C++/Simula; goal = reuse; mobile apps.
- ✦ Class = blueprint/user-defined type; object = instance (new); attributes = data members; encapsulation = data + functions in one unit; abstraction = essentials only; inheritance = is-a, reuse, generalisation, derivation.
- ✦ Messages: object.method(). Constructor: on object creation, no return type, default supplied by compiler, public for general use; copy constructor = init from another object (STOCK: F2/default/copy). Destructor needed for dynamic objects; anonymous class → no destructor (keyed).

## 5.8 C++ — Classes, Access Specifiers, Friends, Streams

**(1)** A function contained within a class is a **member function** (Java calls it a method). **In a class the default access is PRIVATE; in a struct it is PUBLIC** — the one substantive difference. The specifiers **private, protected and public may appear in ANY order and ANY number of times**; each governs the members after it.

| Specifier | Who can access |
|---|---|
| **private** | **Members of the same class only** (and friends) — in Java likewise, only other members of that class; **inherited by a derived class but NOT accessible there** — the favourite trap |
| **protected** | Same class + **derived (sub) classes** — the modifier used to reach members in a deriving class |
| **public** | Everyone |

**(2)** A **friend function** is a **non-member granted access to ALL members — public, private and protected**; the keyword is **friend**. Truth check: a friend may access private + protected data (true); a derived class's member function may reach protected but **never private** base data (false) → **only I**.

**(3)** **Inline functions reduce function-call overhead, mainly for SMALL functions** — the body is expanded in place (no call at all); inline is a request. **new creates instances; delete releases; a dynamic array pointer needs delete [] pp;** (matching new[]).

**(4)** **Shallow copy = copying each member individually** to the corresponding member — what the default copy constructor does (pointers get shared; deep copy duplicates the data). **Namespace: organises code into logical groups / defines a scope** for identifiers (std).

**(5)** **Streams**: **<< = insertion operator** (writes formatted data — the bit-shift operator overloaded for **displaying values**); **>> with cin = extraction**. **cin/cout resolve their conversions at EXECUTION time** (keyed). **cin.ignore(x, y) skips the first x characters, stopping if character y appears**. Manipulators: setw, setfill, resetiosflags — **unset() does not exist**. File-stream classes: **fstreambase = common operations, ifstream = input, ofstream = output, fstream = both** (1-iii, 2-iv, 3-ii, 4-i). Random access: **F.seekp((N−1) × sizeof(EMP))** reaches record N. A **wide-character constant begins with L** (L'x'). **main() is a USER-defined function**. The extra information of a thrown exception travels in the exception object and is received in **the catch block** (keyed).

### Exam Points — 5.8

✦ Member function; class default private, struct default public; specifiers in any order, any number of times. private = same class only, inherited but inaccessible; protected = + subclasses; friend = non-member with full access (only statement I).

✦ Inline = small functions, no call overhead. delete [] for new[]. Shallow copy = member-wise (default). Namespace = logical grouping/scope.

✦ << insertion (overloaded shift → display), >> extraction; cin/cout convert at execution time; ignore(x,y); no unset() manipulator; fstream family matching; seekp((N−1)×sizeof(EMP)); wide char = L; main() user-defined; thrown info → catch block.

## 5.9 C++ — Inheritance, Polymorphism, Templates, Virtual Functions

**(1)** **class circle : public point** — point is the BASE, circle the DERIVED; the colon marks inheritance and public its mode; the false statement reverses the direction (circle's members inherited into point). **Public inheritance: public stays public, protected stays protected, private stays locked in the base** (never accessible in any derived class — "becomes public" is the false claim). NVS worked: Name (protected) → **Protected**, Readdata() (public) → **Public**.

**(2)** **Protected inheritance, second-level derived class**: private **No**, protected **Yes**, public **Yes** (public members turned protected are still reachable down the chain).

**(3)** **Inheritance types**: **multiple = ONE derived class from MANY base classes** (asked four times — 'many parents'); multilevel = a chain; hierarchical = one base, many children. C++ allows multiple inheritance directly; **Java achieves it only through interfaces**.

| Overloading | Overriding |
|---|---|
| **Same name, DIFFERENT parameter lists, same class** — the mechanism letting two related functions share a name | **Same signature, DERIVED class** — redefining an inherited method |
| Resolved at **compile time (static binding)** — with **templates**, the two routes to compile-time polymorphism | Resolved at **run time (dynamic binding)** through **VIRTUAL functions** — run-time polymorphism (asked five times) |
| **Return type alone can NEVER distinguish overloads** (the false statement); static vs non-static also cannot | **::** (scope resolution) calls the base version of an overridden function from the derived class |

**(4)** **Pure virtual function** (= 0): has **no body in the base — it is defined only in derived classes**; a class holding at least one is an **ABSTRACT class** and **cannot be instantiated**. **Abstract methods = methods without a body, defined in an abstract class**. The **advantage of an abstract class over an interface: it can provide default implementations** (and hold state); an **interface defines the set of methods an implementing class MUST provide** — a pure contract.

**(5)** **A generic function is created with a TEMPLATE** (compile-time genericity). **Virtual function use implies dynamic binding** — no virtual keyword, static binding.

### Exam Points — 5.9

- ✦ derived : mode base; public inheritance keeps levels; private never accessible in derived (any mode); protected-inheritance 2nd level = No/Yes/Yes; Name protected + Readdata public.
- ✦ Multiple = many bases (C++ direct; Java via interfaces); multilevel = chain. Overloading = compile-time/static (+ templates); return type alone insufficient. Overriding = run-time/dynamic via virtual; :: reaches the base version.
- ✦ Pure virtual = 0: body only in derived; → abstract class, no instantiation; abstract class gives default implementations; interface = mandatory method set; template → generic function; inline cuts call overhead.

## 5.10 Java — JVM, Classes, Exceptions

Java compiles source into **byte code**, which the **JVM (part of Java)** executes — write once, run anywhere. Java is **strongly typed** (every variable's type declared). Declaration: **int x = 10;**. **All Java functions must be members of some class, and member functions are called methods** (both true). **java.lang.Object is the parent of all classes**. **javap disassembles class files**. PATH locates the compiler; SOAP/RESTful are Java web services. **Garbage collection: when all references to an object are gone, its memory is automatically reclaimed** (System.gc() is only a suggestion). Java **primitives**: byte, short, int, long, float, double, char, boolean — **byte is primitive; Array/String/classes are reference types**; **Integer is a WRAPPER class**. Relational operators: <=, >=, !=, == — **=> is invalid**. **BufferedReader is a CHARACTER stream** (Reader/Writer families; the …Stream classes are byte streams).

**(1)** **Structure rules**: NO semicolon after a method header, and methods live INSIDE the class braces — both tested statements false ('neither'). "static public void main(String…abcd)" IS valid (modifier order is free; varargs fine).

**(2)** **Inheritance uses extends** (implements for interfaces); **super refers to the parent class**; a **protected** method is callable in the child → prints; a **private** method is invisible outside its class → compile error. **final class cannot be subclassed (String); final method cannot be overridden** (both). The false statement in the mixed set: "final methods can be overridden." **Interfaces exist to achieve multiple inheritance**; **abstract method: public abstract void method();** — abstract before the return type, semicolon, no body. **Access order: private < default (package) < protected < public**; **ClassLoaders: Bootstrap → Extension → System — the System ClassLoader has no child**; **packages DO provide controlled access** (the denial is the false statement).

**(3)** **Overloading = compile-time (static binding); overriding = run-time (DYNAMIC binding)** — asked twice more here. **this(10) chains constructors**: prints **Parameterized then Default**. **Copy-style constructor** initialises from another object. Casting: Object obj = new String("Hello"); (String) obj → prints **Hello** (real type matches). **Animal worked**: statements **1, 3, 4** true (no one-arg constructor, so new Animal("Cat") fails; null field prints "Animal: null";

method on a null reference → NullPointerException). **Local (inner) class in a method**: valid; may read effectively-final parameters → **Product: 15** on Java 8+ (the source key expected the pre-Java-8 compile error — corrected).

**(4)** **Exceptions**: an exception is the **run-time disruption of normal flow**. A try block can have **many catches but only ONE finally**. Out-of-range index → **java.lang.ArrayIndexOutOfBoundsException**, then finally runs. **Custom exception extends Exception** → caught, message printed, finally runs. The wrapped-rethrow working: inner finally prints, the new Exception("Division by zero not allowed") reaches the GENERAL catch, outer finally prints — option (c). **Integer and String are usable as identifiers** (not keywords) → prints **24I**; boolean x = (a = b) with int/double → **compile error** (type mismatch); nested if-else picks **b is the largest** (4, 8, 6); String concatenation left-to-right: "exam:" + 9 + 9 + 9 = **exam:999**.

### Exam Points — 5.10

- ✦ javac → byte code → JVM (part of Java); strongly typed; functions = class members = methods; Object is the root; javap disassembles; GC reclaims unreferenced objects; byte primitive, Integer wrapper; => invalid; BufferedReader = character stream.
- ✦ No semicolon after method headers; methods inside the class; extends for inheritance; super = parent; protected inherited-callable, private not; final class/method locked; interfaces → multiple inheritance; public abstract void method(); private<default<protected<public; System ClassLoader childless; packages control access.
- ✦ Overload = static, override = dynamic binding; this(10) → Parameterized then Default; (String) cast prints Hello; Animal → 1,3,4; local class → Product: 15 (Java 8+, key corrected).
- ✦ Exception = runtime disruption; many catches, one finally; ArrayIndexOutOfBoundsException; custom extends Exception; wrapped rethrow → general catch + both finally blocks; 24I; boolean x=(a=b) fails; b is the largest; exam:999.

## 5.11 The .NET Corner

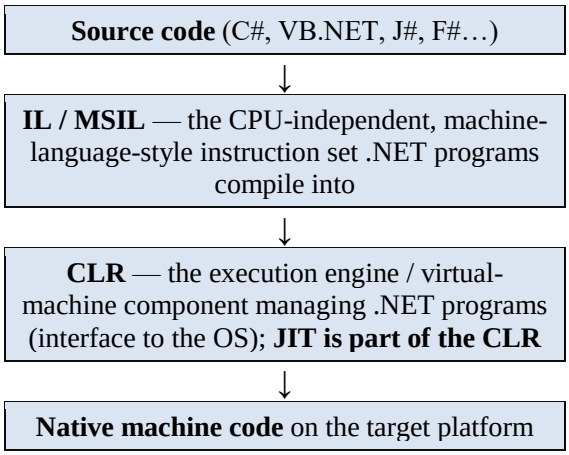



*Fig. 5.1 — The .NET execution pipeline. Java's twin: source → bytecode → JVM.*

**(1)** **.NET was developed by Microsoft, launched in 2002**; today it is **free, cross-platform and open-source**. Its languages include **J#, C#, F# — Pascal is not one**. **VB.NET is an OOP language of the framework but NOT a subset of VB 6.0** (a successor). **ASP.NET applications are compiled** from the framework's reusable components. **The .NET Framework is part of the wider .NET platform** (Linux/macOS/Windows/iOS/Android).

**(2)** **ADO.NET features: interoperability, scalability, maintainability — 'Reachability' is NOT one** (asked twice). **RadioButton** = the VB.NET control for selecting ONE option from a group; **Append** = a **symbolic constant in the file-open function**; **Hashtable** = the collection class accessed by a **unique key**.

**(3)** Two design strays worth keeping: **system design (most abstract) → architectural → detailed (module-level)**; good-language attributes = clarity, abstraction support, ease of verification — **non-portability is NOT one**; when language rules are broken you get **syntax errors**, and **errors are a normal part of a programmer's (and student's) daily life**.

# DSSSB – TGT CS ( PRACTICE KING ) – SUPPORT MATERIAL [ TOPPER'S CHOICE ]

**Exam Points — 5.11**

- ✦ IL = CPU-independent instruction set; CLR = execution engine/VM (JIT inside); Microsoft 2002; free + cross-platform + open source; J#/C#/F# in, Pascal out; VB.NET not a VB6 subset; ASP.NET compiled.
- ✦ ADO.NET ≠ Reachability; RadioButton = single choice; Append = file-open constant; Hashtable = unique-key access; design order system → architectural → detailed; non-portability never a virtue.

## Quick Revision — 35 points

1. C = Ritchie; C++ = Stroustrup; Java = Gosling; FORTRAN = Formula Translation. Tokens = smallest units + compiler's basic elements. Syntax = format, semantics = meaning.
2. Identifiers: letters/digits/underscore, never digit-first; void/static invalid names; volatile a keyword, 'variable' not. int a,b,c = 10 → only c; uninitialised = garbage.
3. Storage classes auto/register/static/extern (no Intern); register = CPU register but garbage default; static & extern default 0; extern = global for all functions.
4. Preprocessor before compilation: #include insertion + escapes + macro expansion (all three). Struct with union = 18 bytes (largest member).
5. ( ) highest; % above + −; = right-associative and lowest; != relational; ?: = 3 operands, right-associative (x?y:a?b:c → c; marks 75 → B); + unary or binary.
6. x−=y+1 = x−y−1; comma keeps the right value (22); precedence worked = 26; 7>=7 → 1; % follows the dividend's sign (1, −1); 4^3 = 7; 5&7 = 5; a=b=c=15 → c+10 = 25.
7. x++ + ++x = 12 (C) / 14 (Java); ++val*++val = 9; 8/++9 = 0; ++m<--k stops at 2000. printf returns its character count (13, 16); 62/062/0x62 → 625098; 017 = 15 (octal).
8. if(cond){}else{}; ladder stops at first true; dangling else → nearest if. switch = multi-choice; falls through till break (FiveDefault; Case 3Outer Default); nested switch legal.
9. while pre-tests (false start = zero runs; while(1)/while(5) infinite — only 0 is false); do-while runs body first, at least once (5; 1 2 3 4 5 corrected; Java 10–12); for's three parts all optional; no foreach in C.
10. break exits, continue skips; nested 3×2 = 6; '1 3' and '2 4 6 8 10'; doubling loop = $\log_2 n + 1$ comparisons; $(i-1)^2$ → 1 0 1 4 9; linked-list traversal A,C,E,X,B.
11. Prototype = counts + types + return only; func(void) with an argument = compile error; nested functions non-standard; actual parameters = values at the call.
12. Address-passing edits the original (swap → 20/10; data[2] → 6); reassigning the pointer parameter is local (20). Recursion = base + self-call; static-i f(1) = 7. Inner blocks shadow (x=11, y=20). SP decreases on CALL. <stdarg.h> variadic.
13. int array[15]; int arr[2][2] = {1,2,3,4} valid; array names not reassignable; arr[i] ≡ *(arr+i); *(*arr+1) = 2 (odd one); sizeof(int arr[5]) = 20.
14. str+7 → World!; S+=2 → SQL; 6+"Hello World" → World; N walk → 1 : 12; str[2][4] = e; %4s → Hell; sprintf returns 13; cgets() console; scanf = fscanf(stdin); struct node { int data; struct node *next; };
15. int *arr[5] = array of pointers; int (*arr)[5] = pointer to array. NULL = references nothing. Static array stack/fixed; dynamic heap/resizable.
16. book.price (dot; arrow for pointers). "wb" binary write; "a+" read+append; empty file opens (Success); fseek SEEK_SET rewinds (42); ftell reports, never moves; fclose.
17. OOP = data first; pillars abstraction + encapsulation + inheritance + polymorphism (all required); data hiding & message passing in, platform independence & global variables out; LISP/Java/C++/Simula (Simula first); goal = reuse.
18. Class = blueprint, user-defined type (data members + member functions); object = instance (new); attributes = data members (name, marks); objects talk by messages (object.method()).
19. Encapsulation = data + functions in one unit; abstraction = essentials without background detail; data hiding = private-access error; extensible = new data types; modularity = independent parts.
20. Constructor: runs on object creation, no return type, default supplied by the compiler, public for use anywhere; copy constructor = init from another object (STOCK: one arg → F2; none → default; object → copy). Destructor: needed for dynamic objects; anonymous class → no destructor (keyed).
21. Inheritance = is-a, reusability, generalisation; Animal → Cat; derivation makes the child; sub + super + reusability are its features.


## DSSSB – TGT COMPUTER SCIENCE ( PRACTICE KING ) BOOK

Call/Message: 7011121290 Available @ Amazon: https://amzn.in/d/07E04o9B

22. C++: member function; class default private, struct default public; specifiers any order/any count; private = same class only, inherited but not accessible; protected reaches subclasses; friend = non-member with access to all three levels (only statement I).
23. Inline = small functions, call removed; delete [] pp for new[]; shallow copy = member-wise default; namespace = scope/logical grouping; wide char L'x'; main() user-defined.
24. << insertion (overloaded shift, displays), >> extraction; cin/cout conversions at execution time; ignore(x,y) = skip x till y; manipulators setw/setfill/resetiosflags (no unset); fstreambase/if/of/fstream = common/in/out/both; seekp((N−1)×sizeof(EMP)); thrown info lands in the catch block (keyed).
25. public inheritance: public→public, protected→protected, private locked (never accessible; Name protected, Readdata public); protected inheritance 2nd level = No/Yes/Yes.
26. Multiple inheritance = many bases for one child (C++ direct, Java via interfaces); multilevel = chain; hierarchical = one base many children.
27. Overloading: same name, different parameters, compile-time/static binding (templates too); return type alone never distinguishes. Overriding: same signature in the derived class, run-time/dynamic binding via virtual; :: calls the base version.
28. Pure virtual (= 0) → defined only in derived classes → abstract class, cannot instantiate; abstract methods have no body; abstract class > interface for default implementations; interface = the mandatory-method contract; template → generic function.
29. Java: javac → byte code → JVM (of Java); strongly typed; int x = 10; all functions are class methods; java.lang.Object at the root; javap disassembles; GC frees unreferenced objects; byte primitive, Integer wrapper; => invalid; BufferedReader = character stream.
30. Java structure: no semicolon after method headers, methods inside braces; extends; super = parent; protected callable in child, private locked; final class (String) unsubclassable, final method unoverridable; interfaces give multiple inheritance; public abstract void method();
31. Access private < default < protected < public; ClassLoaders Bootstrap → Extension → System (System childless); packages DO give access control.
32. this(10) → Parameterized then Default; (String) cast on a real String → Hello; Animal statements 1, 3, 4; local class → Product: 15 on Java 8+ (source key pre-8, corrected); Integer/String as identifiers → 24I; boolean x = (a = b) → compile error.
33. Java exceptions: runtime disruption; many catches, ONE finally; ArrayIndexOutOfBoundsException named; custom extends Exception; wrapped rethrow → inner finally, general catch, outer finally; nested if → b is the largest; "exam:"+9+9+9 = exam:999.
34. .NET: source → IL (CPU-independent) → CLR (execution engine; JIT inside) → native. Microsoft 2002; free/cross-platform/open-source; J#/C#/F# (no Pascal); VB.NET not a VB6 subset; ASP.NET compiled.
35. ADO.NET ≠ Reachability (×2); RadioButton = one of many; Append = file-open symbolic constant; Hashtable = unique-key collection; design: system → architectural → detailed; non-portability never a good-language trait; rule-breaking = syntax error; errors are normal in learning.